SOKOINE UNIVERSITY OF AGRICULTURE

School of Engineering and Technology
Department of Agricultural Engineering

AGE 219: Basics of Computer Programming

Lecture Compendium 5: Functions and Modular Programming — Building Reusable Engineering Tools

---

Prepared by:

Dr. Kadeghe Fue, PhD, P.Eng (T)

Target Audience: Engineering Students (Agricultural, Irrigation, Post-Harvest, and related fields)
Primary Tools: Python 3.x and Google Colaboratory
Location: Morogoro, Tanzania

This lecture introduces the concept of abstraction and modularity. In engineering, we don't build a tractor as a single, indivisible block; we build it as a collection of modules (engine, transmission, hydraulics). Programming follows the same principle. By mastering functions, you will learn how to build "digital tools" that can be used repeatedly across different projects.

Contents

1 Introduction: The "Black Box" Theory of Engineering **2**
1.1 Why use Functions in Engineering? . . . 2

2 Defining and Calling Functions **2**
2.1 Basic Syntax . . . 2
2.2 Parameters and Arguments . . . 2

3 The `return` Statement: Getting Data Out **3**

4 Function Arguments: Advanced Features **3**
4.1 Default Parameters . . . 3
4.2 Keyword Arguments . . . 3

5 Variable Scope: Local vs. Global **4**
5.1 Local Variables . . . 4
5.2 Global Variables . . . 4

6 The Python `math` Module **4**

7 Case Study 1: Agricultural Implement Draft Force **4**
7.1 Python Implementation . . . 5

8 Case Study 2: Daily Reference Evapotranspiration (ET_o) **5**
8.1 Python Implementation . . . 6

9 Docstrings and Documentation Standards **6**
9.1 The Google/Standard Format . . . 6

10 Debugging Functions: Common Pitfalls **6**
10.1 1. Forgetting the `return` . . . 6
10.2 2. Parameter Mismatch . . . 6
10.3 3. Modifying Mutable Globals . . . 7

11 Summary **7**

12 Glossary of Terms **8**

13 Extended Self-Assessment Exercises **9**
13.1 Exercise 1: Irrigation Pipe Flow Velocity . . . 9
13.2 Exercise 2: Tractor Fuel Efficiency Converter . . . 9
13.3 Exercise 3: Soil Moisture Category Function . . . 9
13.4 Exercise 4: Mathematical Optimization . . . 9

1 Introduction: The ”Black Box” Theory of Engineering

In agricultural engineering, we often work with ”Black Boxes.” For example, a diesel engine takes in fuel and air (Inputs), performs combustion (Process), and produces torque at the flywheel (Output). As an engineer using that engine, you don’t necessarily need to re-derive the laws of thermodynamics every time you turn the key; you simply need to know how to provide the correct inputs and what to expect as output.

In programming, a **Function** is the ”Black Box.” It is a self-contained block of code that performs a specific task.

1.1 Why use Functions in Engineering?

- **Eliminating Redundancy:** If you have a complex formula for calculating ”Total Dynamic Head” in a pump system, you shouldn’t type that formula 50 times in your script. You write a function once and ”call” it 50 times.
- **Modularity:** Breaking a large program (like a full-farm simulator) into small, manageable functions makes it easier to debug and maintain.
- **Abstraction:** You can use functions written by other engineers (in libraries like NumPy or SciPy) without needing to understand the thousands of lines of code inside them.
- **Collaboration:** In a large engineering firm, one engineer can write the ”Soil Mechanics” module while another writes the ”Irrigation” module. Functions allow these modules to interact cleanly.

2 Defining and Calling Functions

To create a function in Python, we use the `def` keyword (short for define), followed by the function name, parentheses, and a colon.

2.1 Basic Syntax

```
def greet_farmer():
    """This function prints a simple greeting."""
    print("Good morning, Sokoine Farm!")

# Calling the function
greet_farmer()
```

2.2 Parameters and Arguments

Most engineering functions require data to work with.

- **Parameters:** The variables listed inside the parentheses in the function *definition*. They act as placeholders.
- **Arguments:** The actual values sent to the function when it is *called*.

```
# 'area_ha' and 'yield_tons' are PARAMETERS
def calculate_productivity(area_ha, yield_tons):
    productivity = yield_tons / area_ha
    print(f"Productivity: {productivity} tons/ha")
```

```
5
6 # 50.0 and 200.0 are ARGUMENTS
7 calculate_productivity(50.0, 200.0)
```

3 The `return` Statement: Getting Data Out

Many students confuse `print()` with `return`.

- `print()` simply shows a value on the screen. It is for human observation.
- `return` sends a value back to the part of the program that called the function. This allows the result to be used in further calculations.

Engineering Rule of Thumb

A well-designed engineering function should usually **return** a value, not just print it. This makes the function much more versatile for complex workflows.

```
1 def calculate_area(radius):
2     pi = 3.14159
3     area = pi * (radius ** 2)
4     return area # Send the result back
5
6 # We can store the result in a variable for later use
7 my_silo_area = calculate_area(3.5)
8 total_volume = my_silo_area * 10 # Using the returned value
9 print(f"Total Volume: {total_volume} m3")
```

4 Function Arguments: Advanced Features

Engineering systems often have ”standard” operating conditions that only change occasionally. Python handles this through different types of arguments.

4.1 Default Parameters

You can provide a default value to a parameter. If the user doesn’t provide an argument for it, Python uses the default.

```
1 # Defaulting gravity to 9.81 m/s2
2 def calculate_potential_energy(mass_kg, height_m, g=9.81):
3     return mass_kg * g * height_m
4
5 # Call with default g
6 energy1 = calculate_potential_energy(100, 10)
7
8 # Call with a specific g (e.g., if working on a high-precision lab model)
9 energy2 = calculate_potential_energy(100, 10, 9.806)
```

4.2 Keyword Arguments

When a function has many parameters, it is easy to forget the order. Keyword arguments allow you to ignore the order by specifying the parameter name.

```python
def pump_spec(flow_rate, head, efficiency):
    print(f"Pumping at {flow_rate}L/s with {head}m head.")

# Order doesn't matter here
pump_spec(efficiency=0.85, head=40.0, flow_rate=15.0)
```

5 Variable Scope: Local vs. Global

As an engineer, you must manage how variables are used across your program to avoid "side effects" (where one part of the program accidentally changes a variable used by another part).

5.1 Local Variables

Variables defined inside a function are **Local**. They only exist while the function is running. Once the function finishes, that memory is cleared.

5.2 Global Variables

Variables defined in the main part of your script (outside any function) are **Global**. They can be read from anywhere, but modifying them inside a function requires the `global` keyword (though this is generally discouraged in good engineering practice).

```python
system_status = "OK" # Global

def run_test():
    test_val = 100 # Local
    print(f"Status inside: {system_status}")

run_test()
# print(test_val) # ERROR: test_val does not exist outside the function
```

6 The Python `math` Module

Engineering requires more than just basic addition and subtraction. We need trigonometry, logarithms, and constants. Python provides these through the `math` module.

```python
import math

# Using constants
print(f"The value of Pi is: {math.pi}")

# Using functions
angle_deg = 45
angle_rad = math.radians(angle_deg)
print(f"Sine of 45 deg: {math.sin(angle_rad)}")

# Square roots and logs
print(f"Sqrt of 144: {math.sqrt(144)}")
print(f"Log base 10 of 1000: {math.log10(1000)}")
```

7 Case Study 1: Agricultural Implement Draft Force

Engineering Context: The draft force (D) required to pull a tillage implement through soil is calculated based on ASABE (American Society of Agricultural and Biological Engineers) standards.

The simplified formula is:

$$D = F_i[A + B(S) + C(S^2)]WT$$

Where:

- F_i: Soil texture adjustment factor.
- A, B, C: Machine-specific parameters.
- S: Speed (km/h).
- W: Width of implement (m).
- T: Depth (cm).

7.1 Python Implementation

```python
def calculate_plow_draft(width_m, depth_cm, speed_kmh, soil_type="medium"):
    """
    Calculates draft force for a moldboard plow.
    Soil types: 'fine', 'medium', 'coarse'
    """
    # Define soil factors based on texture
    factors = {
        "fine": 1.2,
        "medium": 1.0,
        "coarse": 0.7
    }

    # Check if soil type exists in our database, default to medium if not
    fi = factors.get(soil_type, 1.0)

    # Machine constants for a standard moldboard plow
    A, B, C = 652, 0, 5.1

    # Formula application
    draft_force_newtons = fi * (A + (B * speed_kmh) + (C * (speed_kmh ** 2))) * width_m *
     depth_cm

    return draft_force_newtons

# Example Usage
force = calculate_plow_draft(width_m=3.0, depth_cm=20.0, speed_kmh=6.0, soil_type="fine")
print(f"Required Draft Force: {force:.2f} Newtons")
```

8 Case Study 2: Daily Reference Evapotranspiration (ET_o)

Engineering Context: In irrigation engineering, ET_o is vital for water budgeting. While the full Penman-Monteith equation is complex, the Hargreaves-Samani method is a common modular alternative using only temperature data.

$$ET_o = 0.0023 \cdot R_a \cdot (T_{mean} + 17.8) \cdot \sqrt{T_{max} - T_{min}}$$

Where R_a is extraterrestrial radiation (often provided as a monthly constant).

8.1 Python Implementation

```
def calculate_hargreaves_et0(t_min, t_max, ra):
    """
    Calculates ET0 using Hargreaves-Samani method.
    t_min/t_max: Celsius
    ra: Extraterrestrial radiation (mm/day)
    """
    t_mean = (t_min + t_max) / 2
    temp_range = t_max - t_min

    et0 = 0.0023 * ra * (t_mean + 17.8) * math.sqrt(temp_range)

    return et0

# Example for a day in Morogoro
daily_et = calculate_hargreaves_et0(t_min=20.0, t_max=32.0, ra=15.5)
print(f"Estimated ET0: {daily_et:.2f} mm/day")
```

9 Docstrings and Documentation Standards

In professional engineering, code is read more often than it is written. You should always document your functions using **Docstrings** (multi-line strings placed immediately under the function definition).

9.1 The Google/Standard Format

```
def convert_hectares_to_acres(hectares):
    """
    Converts farm area from metric to imperial units.

    Args:
        hectares (float): Area in Hectares

    Returns:
        float: Area in Acres
    """
    return hectares * 2.47105
```

By using docstrings, you can use the `help()` function in Python to see the documentation for any function: `help(convert_hectares_to_acres)`.

10 Debugging Functions: Common Pitfalls

10.1 1. Forgetting the `return`

If you don't include a `return` statement, the function will return `None` by default. This will cause errors when you try to use the result in math.

10.2 2. Parameter Mismatch

Calling a function with the wrong number of arguments will cause a `TypeError`. *Example:* If a function requires 3 arguments and you only give 2.

10.3 3. Modifying Mutable Globals

If you pass a list to a function and modify it inside, the original list is changed too! This can be a dangerous ”side effect.” (We will cover Lists in depth in Lecture 6).

11 Summary

In this lecture, we have explored the heart of modular programming. You have learned:

- How to define and call **Functions** to encapsulate logic.
- The critical role of **Parameters** (inputs) and **Return Statements** (outputs).
- How to use **Default** and **Keyword** arguments for flexibility.
- The difference between **Local** and **Global** scope and its impact on data integrity.
- How to leverage the `math` **module** for scientific engineering computations.
- How to document your functions using **Docstrings**.

By building reusable functions, you are no longer just writing ”scripts”; you are building an engineering library. In the next lecture, we will look at **Data Structures (Lists and Dictionaries)**, which will allow us to handle large arrays of data efficiently within our functions.

12 Glossary of Terms

- **Abstraction:** The process of hiding the complex internal details of a function and only showing the necessary inputs and outputs to the user.
- **Argument:** The actual value passed to a function when it is called.
- **Docstring:** A string used to document a Python module, class, function, or method.
- **Function:** A reusable block of code that performs a specific task.
- **Global Scope:** The part of the program where variables defined outside of functions can be accessed.
- **Local Scope:** The part of the program restricted to the inside of a function definition.
- **Modular Programming:** A design technique that separates the functionality of a program into independent, interchangeable modules.
- **Parameter:** A variable used in a function definition to represent the data that will be passed into the function.
- **Return Value:** The result that a function sends back to the caller.

13 Extended Self-Assessment Exercises

13.1 Exercise 1: Irrigation Pipe Flow Velocity

The velocity of water (V) in a pipe is calculated as $V = Q/A$, where Q is the flow rate (m^3/s) and A is the cross-sectional area (m^2). **Task:** 1. Write a function `calculate_pipe_area(diameter_mm)` that returns the area in m^2. 2. Write a second function `calculate_velocity(flow_rate_lps, diameter_mm)` that calls the area function and returns the velocity in m/s. 3. Test it with $Q = 25L/s$ and $D = 100mm$.

13.2 Exercise 2: Tractor Fuel Efficiency Converter

European tractors often measure fuel consumption in Liters per Hour (L/h), while American tractors might use Gallons per Hour (GPH). **Task:** Write a function `convert_fuel_units(value, direction="to_imperial")` that can convert in both directions. Use a default argument for `direction`. (1 Gallon = 3.785 Liters).

13.3 Exercise 3: Soil Moisture Category Function

Task: Write a function `get_moisture_status(mc_percent)` that returns a **String** based on the input:

- < 15: "Critical - Wilting Point"
- $15 - 30$: "Optimal"
- > 30: "Saturated - Risk of Runoff"

13.4 Exercise 4: Mathematical Optimization

Task: Use the `math` module to write a function that calculates the "Slant Height" of a conical grain pile given the radius and the angle of repose (in degrees). Use the Pythagorean theorem and trigonometric functions.

End of Lecture 5 Compendium